

12576 - Nisanth Saravanan
L1 .NET & C# Assessment

Digital Receipt Management System

Digital Receipt Management System:

Stores and categorizes digital receipts for personal or business use.

Entities:

Receipt, Vendor, Category, User, ExpenseReport

Please find the assigned case study and perform below actions:

Task:

1. Perform **W3H analysis** on the given Case Study.
2. Create a **Console application** for the given scenario
3. Configure **MySQL** server and create database and tables as per need.
4. Perform all the **CRUD** operations using **ADO.NET** on the given entity.

 1. Add data
 2. Search data by id
 3. Update data by id
 4. Delete data by id
 5. Display all data
 6. Application must be the Responsive and User-friendly menu items.

GitHub Link:

[RelevantzRIX2024/Assessments/.NET C# at 12576 Nisanth · benhar-velankanni/RelevantzRIX2024](#)

W3H Analysis

Project Title: Digital Receipt Management System	
What?	How?
1.What are the Modules/Functionalities needed? A: Modules/Functionalities Required: a) Adding Functionality. b) Display Functionality. c) Search Functionality. d) Delete Functionality. e) Update Functionality. 2. What are the requirements of Adding Functionality? A: Adding Functionality: a) Get User Input. b) Validate user input. c) Inject new data into the SQL Database. 3. What are the requirements of the Display functionality? A: Display Functionality: a) Get data from SQL database. a) Display data. 4. What are the requirements of the Search functionality? A: Search Functionality: a) Get ID as input. b) To display search results. 5. What are the requirements of the Delete Functionality? A: Delete Functionality:	1. Adding Functionality: a) Get User Input: Method 1: Get input via Dedicated Methods. Method 2: Get input via simple Input Statements. Method 3: Get input via File Reader. b) Validate user input: Method 1: Using Input Helper Methods. Method 2: Using While Loop. Method 3: Using a series of Conditional Statements. c) Inject new data into the SQL Database: Method 1: Using MySQL connection and command mechanism. Method 2: Using manual data injection. Method 3: Using File Writer and storing the file in database. 2. Display Functionality: a) Get data from SQL database: Method 1: Using MySQL connection and command mechanism. Method 2: Using manual data collection. Method 3: Using File Reader and a single database file. a) Display data: Method 1: Using simple Output statements. Method 2: Using File Writer. Method 3: Using Dedicated Output Helper Methods.

- a) Get ID as input.
- b) To display deletion results.

6. What are the requirements of the Update Functionality?

A: Update Functionality:

- a) Get User Input.
- b) Validate user input.
- c) Update new data into the SQL Database.

3. Search Functionality:

a) Get ID as input:

Method 1: Get input via Dedicated Methods.

Method 2: Get input via simple Input Statements.

Method 3: Get input via Conditional Parser.

b) To display search results:

Method 1: Using simple Output statements.

Method 2: Using File Writer.

Method 3: Using Dedicated Output Helper Methods.

4. Delete Functionality:

a) Get ID as input:

Method 1: Get input via Dedicated Methods.

Method 2: Get input via simple Input Statements.

Method 3: Get input via Conditional Parser.

b) To display deletion results:

Method 1: Using simple Output statements.

Method 2: Using File Writer.

Method 3: Using Dedicated Output Helper Methods.

5. Update Functionality:

a) Get User Input:

Method 1: Get input via Dedicated Methods.

Method 2: Get input via simple Input Statements.

Method 3: Get input via File Reader.

b) Validate user input:

Method 1: Using Input Helper Methods.

Method 2: Using While Loop.

Method 3: Using a series of Conditional

	Statements. c) Inject new data into the SQL Database: Method 1: Using MySQL connection and command mechanism. Method 2: Using manual data injection. Method 3: Using File Writer and storing the file in database.
1. Adding Functionality: a) Get User Input: Method 1: Get input via Dedicated Methods. A: Efficient and promotes reusability. b) Validate user input: Method 1: Using Input Helper Methods. A: Efficient and promotes reusability. c) Inject new data into the SQL Database: Method 1: Using MySQL connection and command mechanism. A: Simple to implement but is effective for our cause. 2. Display Functionality: a) Get data from SQL database: Method 1: Using MySQL connection and command mechanism. A: Simple to implement but is effective for our cause. a) Display data: Method 1: Using simple Output statements. A: Simple to implement but is effective for our cause. 3. Search Functionality: a) Get ID as input: Method 3: Get input via Conditional	1. Adding Functionality: a) Get User Input: Method 2: Get input via simple Input Statements. Method 3: Get input via File Reader. A: Inefficient and round-about method respectively b) Validate user input: Method 2: Using While Loop. Method 3: Using a series of Conditional Statements. A: Valid methods but less efficient on their own. c) Inject new data into the SQL Database: Method 2: Using manual data injection. Method 3: Using File Writer and storing the file in database. A: Inefficient and round-about method respectively 2. Display Functionality: a) Get data from SQL database: Method 2: Using manual data collection. Method 3: Using File Reader and a single database file. A: Inefficient and round-about method respectively a) Display data: Method 2: Using File Writer. Method 3: Using Dedicated Output Helper Methods.

Parser.

A: Simple to implement but is effective for our cause.

b) To display search results:

Method 1: Using simple Output statements.

A: Simple to implement but is effective for our cause.

4. Delete Functionality:

a) Get ID as input:

Method 3: Get input via Conditional Parser.

A: Simple to implement but is effective for our cause.

b) To display deletion results:

Method 1: Using simple Output statements.

A: Simple to implement but is effective for our cause.

5. Update Functionality:

a) Get User Input:

Method 1: Get input via Dedicated Methods.

A: Efficient and promotes reusability.

b) Validate user input:

Method 1: Using Input Helper Methods.

A: Efficient and promotes reusability.

c) Inject new data into the SQL Database:

Method 1: Using MySQL connection and command mechanism.

A: Simple to implement but is effective for our cause.

A: Inefficient and round-about method respectively

3. Search Functionality:

a) Get ID as input:

Method 1: Get input via Dedicated Methods.

Method 2: Get input via simple Input Statements.

A: Increases the lines of code with no valid justification.

b) To display search results:

Method 2: Using File Writer.

Method 3: Using Dedicated Output Helper Methods.

A: Inefficient and round-about method respectively

4. Delete Functionality:

a) Get ID as input:

Method 1: Get input via Dedicated Methods.

Method 2: Get input via simple Input Statements.

A: Increases the lines of code with no valid justification.

b) To display deletion results:

Method 2: Using File Writer.

Method 3: Using Dedicated Output Helper Methods.

A: Inefficient and round-about method respectively

5. Update Functionality:

a) Get User Input:

Method 2: Get input via simple Input Statements.

Method 3: Get input via File Reader.

A: Inefficient and round-about method

	respectively b) Validate user input: Method 2: Using While Loop. Method 3: Using a series of Conditional Statements. A: Valid methods but less efficient on their own. c) Inject new data into the SQL Database: Method 2: Using manual data injection. Method 3: Using File Writer and storing the file in database. A: Inefficient and round-about method respectively
Why?	Why Not?

CODE

Database Schema

```
CREATE TABLE DIGIRECEIPT (  
  ID INT AUTO_INCREMENT PRIMARY KEY,  
  RECEIPTNAME VARCHAR(50),  
  RECEIPTAMOUNT DECIMAL(10,2),  
  CATEGORY VARCHAR(50),  
  DATE VARCHAR(10),  
  ADDITIONALNOTES VARCHAR(500));
```

Program.cs

```
namespace DigiReceiptApp
{
    public class Program
    {
        public static void Main(String[] args)
        {
            Console.WriteLine("\n=====\\nWELCOME TO OUR APPLICATION.\\n=====");

            string connectionString =
"Server=localhost;User=root;Password=Reset@123;Database=RELEVANTZ;Port=3306;";

            IDigiReceiptService digiReceiptService = new
DigiReceiptService(connectionString);
            DigiReceiptUI digiReceiptUI = new
DigiReceiptUI(digiReceiptService);

            while (true)
            {
                Console.WriteLine("\n=====\\nMENU:\\n=====");

                Console.WriteLine("1. ADD RECEIPT.");
                Console.WriteLine("2. GET RECEIPT BY ID.");
                Console.WriteLine("3. GET ALL RECEIPTS.");
                Console.WriteLine("4. UPDATE RECEIPT.");
                Console.WriteLine("5. DELETE RECEIPT.");
                Console.WriteLine("6. TRUNCATE RECEIPTS.");
                Console.WriteLine("0. EXIT.");
```

```

Console.WriteLine("=====
=====");
    Console.WriteLine("ENTER YOUR CHOICE:");

    if (int.TryParse(Console.ReadLine(), out int choice))
    {
        switch (choice)
        {
            case 1:
                DigiReceiptUI.AddReceiptUI();
                break;
            case 2:
                DigiReceiptUI.GetReceiptByIdUI();
                break;
            case 3:
                DigiReceiptUI.GetAllReceiptsUI();
                break;
            case 4:
                DigiReceiptUI.UpdateDigiReceiptUI();
                break;
            case 5:
                DigiReceiptUI.DeleteDigiReceiptUI();
                break;
            case 6:
                DigiReceiptUI.TruncateDigiReceiptUI();
                break;
            case 0:
                Console.WriteLine("\
n=====\\
nTHANK YOU FOR USING OUR APPLICATION.\\
n=====");
                return;
            default:
                Console.WriteLine("\
n=====\\

```

```

nINVALID CHOICE. PLEASE TRY AGAIN.\
n=====");
        break;
    }
}
else
{
    Console.WriteLine("\
n=====\\
nINVALID INPUT. PLEASE TRY AGAIN.\
n=====");
}
}
}
}
}
}
}

```

DigiReceipt.cs

```
using System.Text.RegularExpressions;
using MySql.Data.MySqlClient;

namespace DigiReceiptApp
{
    public class DigiReceipt
    {
        public int? ID { get; set; }
        public string? RECEIPTNAME { get; set; }
        public decimal? RECEIPTAMOUNT { get; set; }
        public string? CATEGORY { get; set; }
        public string? DATE { get; set; }
        public string? ADDITIONALNOTES { get; set; }
    }

    public interface IDigiReceiptService
    {
        void AddReceipt(DigiReceipt digiReceipt);
        DigiReceipt GetDigiReceiptByID(int id);
        List<DigiReceipt> GetAllDigiReceipts();
        bool UpdateDigiReceipt(int id, DigiReceipt digiReceipt);
        bool DeleteDigiReceipt(int id);
        bool TruncateDigiReceipt();
    }

    public class DigiReceiptService : IDigiReceiptService
    {
        private readonly string _connectionString;

        public DigiReceiptService(string connectionString)
        {
            _connectionString = connectionString;
        }

        public void AddReceipt(DigiReceipt digiReceipt)
```

```

    {
        try
        {
            using (var connection = new
MySQLConnection(_connectionString))
            {
                connection.Open();
                using (var command = new MySqlCommand("INSERT INTO
DIGIRECEIPT (RECEIPTNAME, RECEIPTAMOUNT, CATEGORY, DATE,
ADDITIONALNOTES) VALUES (@name, @amount, @category, @date,
@notes)", connection))
                {
                    command.Parameters.AddWithValue("@name",
digiReceipt.RECEIPTNAME);
                    command.Parameters.AddWithValue("@amount",
digiReceipt.RECEIPTAMOUNT);
                    command.Parameters.AddWithValue("@category",
digiReceipt.CATEGORY);
                    command.Parameters.AddWithValue("@date",
digiReceipt.DATE);
                    command.Parameters.AddWithValue("@notes",
digiReceipt.ADDITIONALNOTES);
                    command.ExecuteNonQuery();
                }
            }
        }
        catch (Exception ex)
        {
            throw new Exception($"Error adding receipt: {ex.Message}", ex);
        }
    }

    public DigiReceipt GetDigiReceiptByID(int id)
    {
        try
        {

```

```

        using (var connection = new
MySQLConnection(_connectionString))
        {
            connection.Open();
            using (var command = new MySqlCommand("SELECT * FROM
DIGIRECEIPT WHERE ID = @id", connection))
            {
                command.Parameters.AddWithValue("@id", id);
                using (var reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        return new DigiReceipt
                        {
                            ID = reader.GetInt32("ID"),
                            RECEIPTNAME = reader.GetString("RECEIPTNAME"),
                            RECEIPTAMOUNT =
reader.GetDecimal("RECEIPTAMOUNT"),
                            CATEGORY = reader.GetString("CATEGORY"),
                            DATE = reader.GetString("DATE"),
                            ADDITIONALNOTES =
reader.GetString("ADDITIONALNOTES")
                        };
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        throw new Exception($"Error getting receipt by ID:
{ex.Message}", ex);
    }
    return null;
}

```

```

public List<DigiReceipt> GetAllDigiReceipts()

```

```

{
    var digiReceipts = new List<DigiReceipt>();

    try
    {
        using (var connection = new
MySQLConnection(_connectionString))
        {
            connection.Open();
            using (var command = new MySqlCommand("SELECT * FROM
DIGIRECEIPT", connection))
            {
                using (var reader = command.ExecuteReader())
                {
                    if (reader.HasRows)
                    {
                        while (reader.Read())
                        {
                            digiReceipts.Add(new DigiReceipt
                            {
                                ID = reader.GetInt32("ID"),
                                RECEIPTNAME = reader.GetString("RECEIPTNAME"),
                                RECEIPTAMOUNT =
reader.GetDecimal("RECEIPTAMOUNT"),
                                CATEGORY = reader.GetString("CATEGORY"),
                                DATE = reader.GetString("DATE"),
                                ADDITIONALNOTES =
reader.GetString("ADDITIONALNOTES")
                            });
                        }

                        return digiReceipts;
                    }
                    else
                    {
                        return null;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}
}
catch (Exception ex)
{
    throw new Exception($"Error getting all receipts: {ex.Message}",
ex);
}
}

```

```

public bool UpdateDigiReceipt(int id, DigiReceipt digiReceipt)
{
    try
    {
        using (var connection = new
 MySqlConnection(_connectionString))
        {
            connection.Open();
            using (var command = new MySqlCommand("UPDATE
DIGIRECEIPT SET RECEIPTNAME = @name, RECEIPTAMOUNT = @amount,
CATEGORY = @category, DATE = @date, ADDITIONALNOTES = @notes
WHERE ID = @id", connection))
            {
                command.Parameters.AddWithValue("@id", id);
                command.Parameters.AddWithValue("@name",
digiReceipt.RECEIPTNAME);
                command.Parameters.AddWithValue("@amount",
digiReceipt.RECEIPTAMOUNT);
                command.Parameters.AddWithValue("@category",
digiReceipt.CATEGORY);
                command.Parameters.AddWithValue("@date",
digiReceipt.DATE);
                command.Parameters.AddWithValue("@notes",
digiReceipt.ADDITIONALNOTES);
                int rowsAffected = command.ExecuteNonQuery();
                return rowsAffected > 0;
            }
        }
    }
}

```

```

        }
    }
}
catch (Exception ex)
{
    throw new Exception($"Error updating receipt: {ex.Message}",
ex);
}
}

```

```

public bool DeleteDigiReceipt(int id)
{
    try
    {
        using (var connection = new
 MySqlConnection(_connectionString))
        {
            connection.Open();
            using (var command = new MySqlCommand("DELETE FROM
DIGIRECEIPT WHERE ID = @id", connection))
            {
                command.Parameters.AddWithValue("@id", id);
                int rowsAffected = command.ExecuteNonQuery();
                return rowsAffected > 0;
            }
        }
    }
    catch (Exception ex)
    {
        throw new Exception($"Error deleting receipt: {ex.Message}",
ex);
    }
}

```

```

public bool TruncateDigiReceipt()
{
    try

```

```

        {
            using (var connection = new
MySQLConnection(_connectionString))
            {
                connection.Open();
                using (var command = new MySqlCommand("TRUNCATE TABLE
DIGIRECEIPT", connection))
                {
                    int rowsAffected = command.ExecuteNonQuery();
                    return rowsAffected > 0;
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"\\
n=====\\nERROR
IN TRUNCATING RECEIPT: {ex.Message} \\
n=====");
        }
        return false;
    }

}

```

```

public class DigiReceiptUI
{
    public static IDigiReceiptService _digiReceiptService;

    public DigiReceiptUI(IDigiReceiptService digiReceiptService)
    {
        _digiReceiptService = digiReceiptService;
    }

    public static void AddReceiptUI()
    {

```

```

        Console.WriteLine("\n=====
RECEIPT:\n=====");

```

```

        var name = GetValidName();
        Console.WriteLine();
        var amount = GetValidAmount();
        Console.WriteLine();
        var category = GetValidCategory();
        Console.WriteLine();
        var date = GetValidDate();
        Console.WriteLine();
        var notes = GetValidNotes();
        Console.WriteLine();

```

```

Console.WriteLine("=====
=====");

```

```

        var digiReceipt = new DigiReceipt
        {
            RECEIPTNAME = name,
            RECEIPTAMOUNT = amount,
            CATEGORY = category,
            DATE = date,
            ADDITIONALNOTES = notes
        };

```

```

        try
        {
            _digiReceiptService.AddReceipt(digiReceipt);
            Console.WriteLine("\n=====
RECEIPT ADDED SUCCESSFULLY.\n=====");
        }

```

```

        catch (Exception ex)
        {
            Console.WriteLine($"\\
n=====\\
nERROR: {ex.Message} \\
n=====");
        }
    }

    public static void GetAllReceiptsUI()
    {
        Console.WriteLine("\\
n=====\\nGET
ALL RECEIPTS:\\
n=====");
        Console.WriteLine();

        try
        {
            var digiReceipts = _digiReceiptService.GetAllDigiReceipts();
            if (digiReceipts != null)
            {
                foreach (var digiReceipt in digiReceipts)
                {
                    Console.WriteLine($"Receipt ID: {digiReceipt.ID}");
                    Console.WriteLine($"Receipt Name:
{digiReceipt.RECEIPTNAME}");
                    Console.WriteLine($"Receipt Amount:
{digiReceipt.RECEIPTAMOUNT}");
                    Console.WriteLine($"Category of Expenditure:
{digiReceipt.CATEGORY}");
                    Console.WriteLine($"Date of Expenditure:
{digiReceipt.DATE}");
                    if (digiReceipt.ADDITIONALNOTES != "")
                    {
                        Console.WriteLine($"Additional Notes:
{digiReceipt.ADDITIONALNOTES}");

```

```

    }
    else
    {
        Console.WriteLine($"Additional Notes: N/A");
    }

```

```

Console.WriteLine("=====
=====");
    }

```

```

    }
    else
    {
        Console.WriteLine("\n=====
RECEIPTS FOUND.\n=====");
    }

```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine($"=====
nERROR: {ex.Message} \n=====");
    }
}

```

```

    public static void GetReceiptByIdUI()
    {
        Console.WriteLine("\n=====
RECEIPT BY ID:\n=====");
        Console.WriteLine();
    }

```

```

    if (int.TryParse(Console.ReadLine(), out int id))
    {

```

```

try
{
    var digiReceipt = _digiReceiptService.GetDigiReceiptByID(id);
    if (digiReceipt != null)
    {
        Console.WriteLine($"Receipt ID: {digiReceipt.ID}");
        Console.WriteLine($"Receipt Name:
{digiReceipt.RECEIPTNAME}");
        Console.WriteLine($"Receipt Amount:
{digiReceipt.RECEIPTAMOUNT}");
        Console.WriteLine($"Category of Expenditure:
{digiReceipt.CATEGORY}");
        Console.WriteLine($"Date of Expenditure:
{digiReceipt.DATE}");
        if (digiReceipt.ADDITIONALNOTES != "")
        {
            Console.WriteLine($"Additional Notes:
{digiReceipt.ADDITIONALNOTES}");
        }
        else
        {
            Console.WriteLine($"Additional Notes: N/A");
        }

        Console.WriteLine("=====
=====");
    }
    else
    {
        Console.WriteLine("\n
=====\\nNO
RECEIPT FOUND WITH THE GIVEN ID.\\n
=====");
    }
}
catch (Exception ex)
{

```

```

        Console.WriteLine($"\\
n=====\\
nERROR: {ex.Message} \\
n=====");
    }
}
else
{
    Console.WriteLine("\\
n=====\\
nINVALID ID FORMAT. PLEASE ENTER A NUMBER.\\
n=====");
}
}

public static void UpdateDigiReceiptUI()
{
    Console.WriteLine("\\
n=====\\
nUPDATE RECEIPT: ENTER RECIEPT ID:\\
n=====");
    Console.WriteLine();

    if (int.TryParse(Console.ReadLine(), out int id))
    {
        var digiReceipt = _digiReceiptService.GetDigiReceiptByID(id);
        if (digiReceipt != null)
        {
            Console.WriteLine("\\
n=====\\nENTER
NEW RECEIPT DETAILS:\\
n=====");
            var name = GetValidName();
            Console.WriteLine();
            var amount = GetValidAmount();
            Console.WriteLine();
            var category = GetValidCategory();

```

```

Console.WriteLine();
var date = GetValidDate();
Console.WriteLine();
var notes = GetValidNotes();
Console.WriteLine();

```

```

Console.WriteLine("=====
=====");

```

```

    digiReceipt.RECEIPTNAME = name;
    digiReceipt.RECEIPTAMOUNT = amount;
    digiReceipt.CATEGORY = category;
    digiReceipt.DATE = date;
    digiReceipt.ADDITIONALNOTES = notes;

```

```

    try
    {
        bool result = _digiReceiptService.UpdateDigiReceipt(id,
digiReceipt);
        if (result)
        {
            Console.WriteLine("\
n=====\\
nRECEIPT UPDATED SUCCESSFULLY.\\
n=====");
        }
        else
        {
            Console.WriteLine("\
n=====\\
nRECEIPT UPDATION FAILED.\\
n=====");
        }
    }
    catch (Exception ex)
    {

```

```

        Console.WriteLine($"\\
n=====\\
nERROR: {ex.Message} \\
n=====");
    }
}
else
{
    Console.WriteLine("\\
n=====\\nNO
RECEIPT FOUND WITH THE GIVEN ID.\\
n=====");
}
}
}

```

```

public static void DeleteDigiReceiptUI()
{
    Console.WriteLine("\\
n=====\\
nDELETE RECEIPT: ENTER RECIEPT ID:\\
n=====");

    if (int.TryParse(Console.ReadLine(), out int id))
    {
        try
        {
            bool result = _digiReceiptService.DeleteDigiReceipt(id);
            if (result)
            {
                Console.WriteLine("\\
n=====\\
nRECEIPT DELETED SUCCESSFULLY.\\
n=====");
            }
            else
            {

```

```

        Console.WriteLine("\n=====\\nRECEIPT DELETION FAILED.\\n=====");
    }
}
catch (Exception ex)
{
    Console.WriteLine($"\\n=====\\nERROR: {ex.Message} \\n=====");
}
}
else
{
    Console.WriteLine("\n=====\\nINVALID ID FORMAT. PLEASE ENTER A NUMBER.\\n=====");
}
}

```

```

public static void TruncateDigiReceiptUI()
{
    Console.WriteLine("\n=====\\nTRUNCATE RECEIPT:\\n=====");
}

```

```

    Console.Write("Confirmation: All existing data will be deleted. Press Y to continue: ");
    string? choice = Console.ReadLine().ToUpper();

    if (choice == "Y")
    {
        try

```

```

        {
            _digiReceiptService.TruncateDigiReceipt();
            Console.WriteLine("\n=====
nRECEIPT TABLE TRUNCATED SUCCESSFULLY.\n=====");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"
n=====
nERROR: {ex.Message} \n=====");
        }
    }
    else
    {
        Console.WriteLine("\n=====
nOPERATION CANCELLED.\n=====");
    }
}

```

```

private static string GetValidName()
{
    while (true)
    {
        Console.WriteLine("Enter a valid receipt name:");
        var name = Console.ReadLine();
        if (!string.IsNullOrEmpty(name))
        {
            return name;
        }
        Console.WriteLine("Invalid name. Please try again.");
    }
}

```

```
private static decimal GetValidAmount()
{
    while (true)
    {
        Console.WriteLine("Enter a valid amount (INR:");
        if (decimal.TryParse(Console.ReadLine(), out decimal amount))
        {
            if (amount >= 0)
            {
                return Math.Round(amount, 2);
            }
            else
            {
                Console.WriteLine("Invalid amount. Please try again.");
                continue;
            }
        }
        else
        {
            Console.WriteLine("Invalid amount. Please try again.");
        }
    }
}
```

```
private static string GetValidCategory()
{
    while (true)
    {
        Console.WriteLine("Enter a valid category:");
        var category = Console.ReadLine();
        if (!string.IsNullOrEmpty(category))
        {
            return category;
        }
        Console.WriteLine("Invalid category. Please try again.");
    }
}
```

```
}
```

```
private static string GetValidDate()
{
    while (true)
    {
        Console.WriteLine("Enter a valid date (DD/MM/YYYY):");
        var date = Console.ReadLine();
        string dateRegex = @"^(0[1-9]|[12][0-9]|3[01])\V(0[1-9]|1[0-2])\V[0-9]{4}$";
        if (date == null)
        {
            Console.WriteLine("Invalid date. Please try again.");
            continue;
        }
        if (Regex.IsMatch(date, dateRegex))
        {
            return date;
        }
        Console.WriteLine("Invalid date. Please try again.");
    }
}
```

```
private static string GetValidNotes()
{
    while (true)
    {
        Console.WriteLine("Enter a valid notes:");
        var notes = Console.ReadLine();
        if (!string.IsNullOrEmpty(notes))
        {
            return notes;
        }
        else
        {
            answerme1:
            Console.WriteLine("No Additional Notes? (Y/N)");
```

```
var answer = Console.ReadLine();
if (answer == null)
{
    Console.WriteLine("Invalid answer. Please try again.");
    goto answerme1;
}
if (answer.ToUpper() == "Y")
{
    return "";
}
}
}
}
}
```

SNIPS

```
mysql> DESCRIBE DIGIRECEIPT;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| ID             | int           | NO   | PRI | NULL    | auto_increment |
| RECEIPTNAME    | varchar(50)   | YES  |     | NULL    |                |
| RECEIPTAMOUNT  | decimal(10,2) | YES  |     | NULL    |                |
| CATEGORY       | varchar(50)   | YES  |     | NULL    |                |
| DATE           | varchar(10)   | YES  |     | NULL    |                |
| ADDITIONALNOTES | varchar(500)  | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Starting Point

```
=====
WELCOME TO OUR APPLICATION.
=====

=====
MENU:
=====
1. ADD RECEIPT.
2. GET RECEIPT BY ID.
3. GET ALL RECEIPTS.
4. UPDATE RECEIPT.
5. DELETE RECEIPT.
6. TRUNCATE RECEIPTS.
0. EXIT.
=====
ENTER YOUR CHOICE:
█
```

Validations

```
Enter a valid receipt name:
```

```
Invalid name. Please try again.
```

```
Enter a valid receipt name:
```

```
█
```

```
Enter a valid amount (INR):
```

```
Invalid amount. Please try again.
```

```
Enter a valid amount (INR):
```

```
█
```

```
Enter a valid amount (INR):
```

```
-100
```

```
Invalid amount. Please try again.
```

```
Enter a valid amount (INR):
```

```
█
```

```
Enter a valid category:
```

```
Invalid category. Please try again.
```

```
Enter a valid category:
```

```
█
```

```
Enter a valid date (DD/MM/YYYY):
```

```
Invalid date. Please try again.
```

```
Enter a valid date (DD/MM/YYYY):
```

```
█
```

```
Enter a valid date (DD/MM/YYYY):
```

```
12\654\53135
```

```
Invalid date. Please try again.
```

```
Enter a valid date (DD/MM/YYYY):
```

```
Enter a valid notes:
```

```
No Additional Notes? (Y/N)
```

```
n
```

```
Enter a valid notes:
```

```
No Additional Notes? (Y/N)
```

```
y
```

Add Data

```
ENTER YOUR CHOICE:
1

=====
ADD RECEIPT:
=====
Enter a valid receipt name:
Groceries

Enter a valid amount (INR):
450

Enter a valid category:
HouseHold

Enter a valid date (DD/MM/YYYY):
21/05/2025

Enter a valid notes:
Forgot to Buy Carrots.

=====

=====
RECEIPT ADDED SUCCESSFULLY.
=====
```

Reflection

Note: The data is different because I truncated the table prior to documentation.

```
mysql> SELECT * FROM DIGIRECEIPT;
+----+-----+-----+-----+-----+-----+
| ID | RECEIPTNAME | RECEIPTAMOUNT | CATEGORY | DATE | ADDITIONALNOTES |
+----+-----+-----+-----+-----+-----+
| 1 | Groceries | 540.00 | HouseHold | 20/05/2025 | |
| 2 | Electricity Bill | 4500.00 | General | 25/05/2025 | Add Tax Later. |
+----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

List All Data

Note: Despite entering "450" alone, the data is stored as decimal rounded off by 2 in default.

And is there is no additional details given during the adding process, the reflection is N/A.

```
ENTER YOUR CHOICE:
3

=====
GET ALL RECEIPTS:
=====

Receipt ID: 1
Receipt Name: Groceries
Receipt Amount: 450.00
Category of Expenditure: HouseHold
Date of Expenditure: 21/05/2025
Additional Notes: Forgot to Buy Carrots.
=====

Receipt ID: 2
Receipt Name: Electricity Bill
Receipt Amount: 4500.00
Category of Expenditure: General
Date of Expenditure: 12/12/1212
Additional Notes: N/A
=====
```

Search by ID

ENTER YOUR CHOICE:

2

=====

GET RECEIPT BY ID:

=====

2

Receipt ID: 2

Receipt Name: Electricity Bill

Receipt Amount: 4500.00

Category of Expenditure: General

Date of Expenditure: 12/12/1212

Additional Notes: N/A

=====

Update By ID

```
ENTER YOUR CHOICE:
```

```
4
```

```
=====
UPDATE RECEIPT: ENTER RECIEPT ID:
```

```
=====
1
```

```
=====
ENTER NEW RECEIPT DETAILS:
```

```
=====
ENTER NEW RECEIPT DETAILS:
```

```
=====
Enter a valid receipt name:
```

```
Groceries
```

```
Enter a valid amount (INR):
```

```
785
```

```
Enter a valid category:
```

```
General
```

```
Enter a valid date (DD/MM/YYYY):
```

```
25/05/2025
```

```
Enter a valid notes:
```

```
No Additional Notes? (Y/N)
```

```
y
```

```
=====
RECEIPT UPDATED SUCCESSFULLY.
```

Reflection of Update

Note: The additional notes have been modifies to N/A due to no additional notes given during update.

```
ENTER YOUR CHOICE:
3

=====
GET ALL RECEIPTS:
=====

Receipt ID: 1
Receipt Name: Groceries
Receipt Amount: 785.00
Category of Expenditure: General
Date of Expenditure: 25/05/2025
Additional Notes: N/A
=====

Receipt ID: 2
Receipt Name: Electricity Bill
Receipt Amount: 4500.00
Category of Expenditure: General
Date of Expenditure: 12/12/1212
Additional Notes: N/A
=====
```

Deletion By ID

```
ENTER YOUR CHOICE:
```

```
5
```

```
=====
DELETE RECEIPT: ENTER RECIEPT ID:
```

```
=====
1
```

```
=====
RECEIPT DELETED SUCCESSFULLY.
=====
```

Reflection of Deletion

```
ENTER YOUR CHOICE:
```

```
3
```

```
=====
GET ALL RECEIPTS:
```

```
=====
Receipt ID: 2
Receipt Name: Electricity Bill
Receipt Amount: 4500.00
Category of Expenditure: General
Date of Expenditure: 12/12/1212
Additional Notes: N/A
=====
```

Additionally Truncate (For Resetting DB during testing)

```
=====
ENTER YOUR CHOICE:
```

```
6
```

```
=====
TRUNCATE RECEIPT:
```

```
=====
Confirmation: All existing data will be deleted. Press Y to continue: n
```

```
=====
OPERATION CANCELLED.
=====
```

```
ENTER YOUR CHOICE:
```

```
6
```

```
=====
TRUNCATE RECEIPT:
```

```
=====
Confirmation: All existing data will be deleted. Press Y to continue: y
```

```
=====
RECEIPT TRUNCATED SUCCESSFULLY.
=====
```

Event: Failures

Note: Database/Table is now empty.

```
ENTER YOUR CHOICE:
```

```
2
```

```
=====
GET RECEIPT BY ID:
```

```
1
```

```
=====
NO RECEIPT FOUND WITH THE GIVEN ID.
=====
```

```
ENTER YOUR CHOICE:
```

```
3
```

```
=====
GET ALL RECEIPTS:
```

```
=====
NO RECEIPTS FOUND.
=====
```

```
ENTER YOUR CHOICE:
```

```
4
```

```
=====
UPDATE RECEIPT: ENTER RECIEPT ID:
```

```
1
```

```
=====
NO RECEIPT FOUND WITH THE GIVEN ID.
=====
```

ENTER YOUR CHOICE:

5

=====

DELETE RECEIPT: ENTER RECIEPT ID:

=====

1

=====

RECEIPT DELETED FAILED.

=====

More Validations

```
=====
MENU:
=====
```

- ```
1. ADD RECEIPT.
2. GET RECEIPT BY ID.
3. GET ALL RECEIPTS.
4. UPDATE RECEIPT.
5. DELETE RECEIPT.
6. TRUNCATE RECEIPTS.
0. EXIT.
```

```
=====
ENTER YOUR CHOICE:
CAWC
```

```
=====
INVALID INPUT. PLEASE TRY AGAIN.
=====
```

```
ENTER YOUR CHOICE:
2
```

```
=====
GET RECEIPT BY ID:
=====
```

```
dqwd
```

```
=====
INVALID ID FORMAT. PLEASE ENTER A NUMBER.
=====
```

Note: The same validation for ID and Choice inputs carries over, everywhere in this app.